

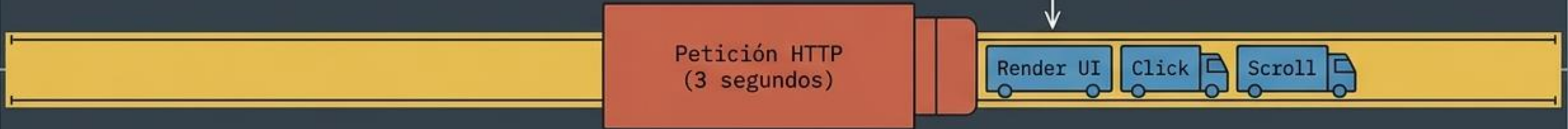
Asincronía en JavaScript y React

Cápsula de Microlearning (Semana 6): Controlando el tiempo, las promesas y las APIs sin bloquear la interfaz.

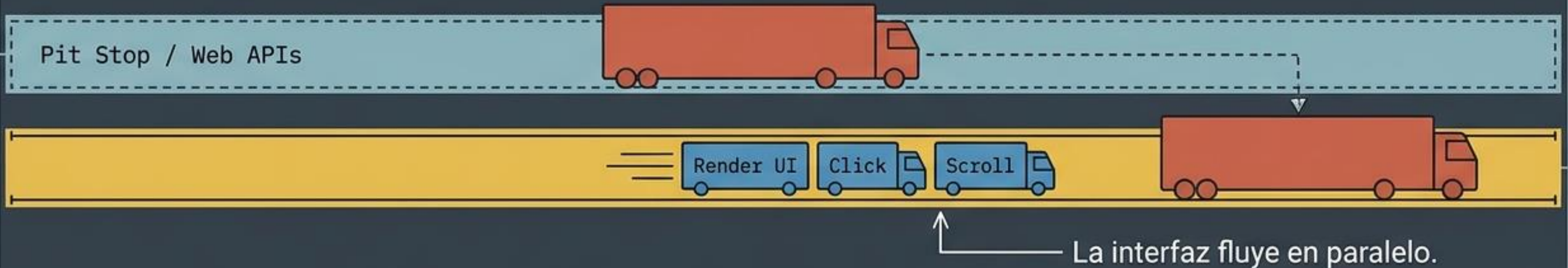


El Problema del Single-Thread: Evitando el Bloqueo

Modelo Síncrono (Bloqueante)

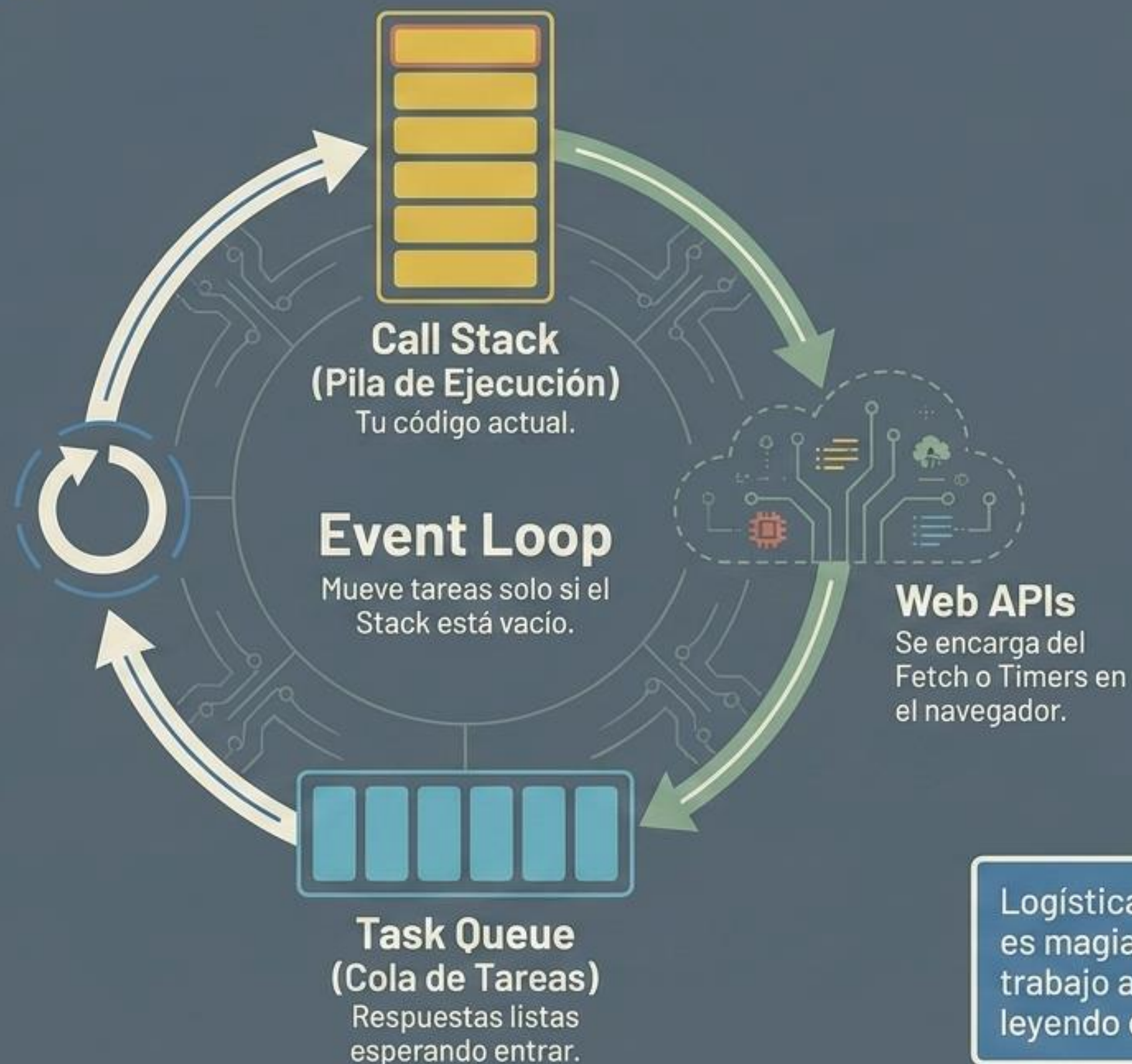


Modelo Asíncrono (No Bloqueante)



Takeaway: JavaScript ejecuta una sola tarea a la vez. La asincronía desvía las tareas pesadas para mantener la interfaz interactiva.

El Motor Oculto: El Event Loop



Logística pura: La asincronía no es magia multihilo. JS encarga el trabajo a las Web APIs y continúa leyendo el código.

El Estándar Moderno: ¿Qué es una Promesa?



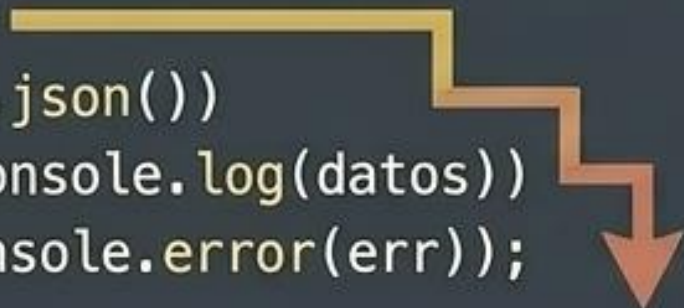
Una Promesa es un objeto que representa la terminación o el fracaso eventual de una operación asíncrona.

Evolución de Sintaxis: then/catch vs async/await

Enfoque Clásico: then / catch

Cascada / Indentación profunda

```
1 fetch('api/datos')  
2   .then(res => res.json())  
3   .then(datos => console.log(datos))  
4   .catch(err => console.error(err));
```



Estándar Moderno: async / await

Plano / Lectura Síncrona

```
1 const res = await fetch('api/datos');  
2 const datos = await res.json();  
3  
4 console.log(datos);
```



async/await es azúcar sintáctico sobre las Promesas.
Pausa la ejecución de la función, no de la UI.

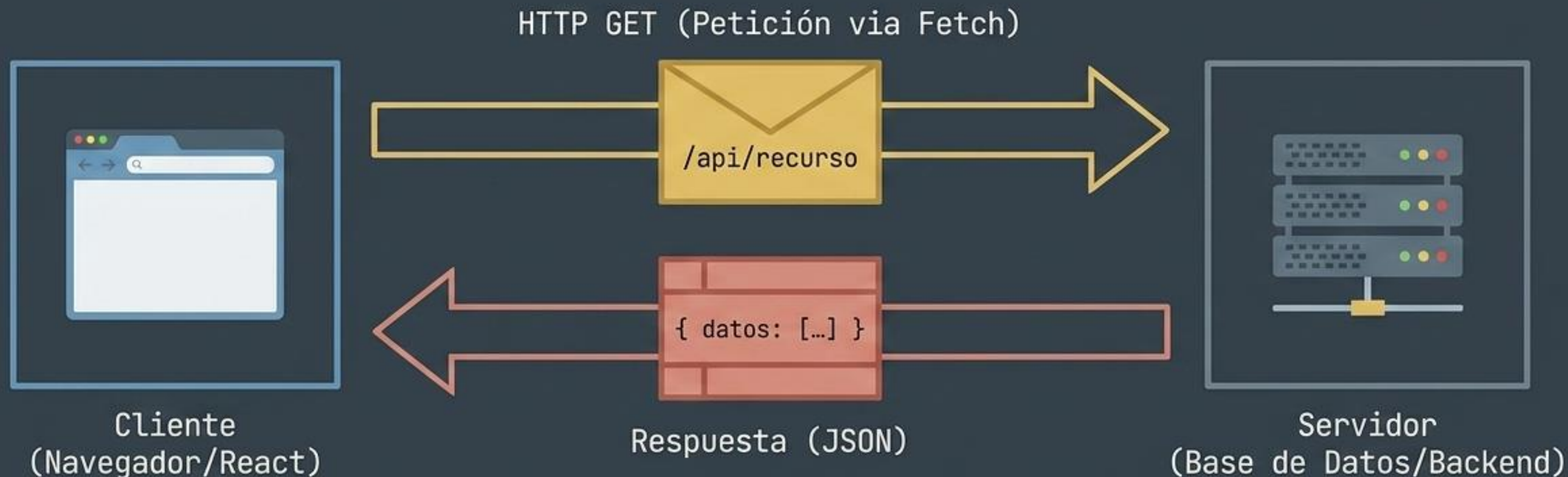
La Regla de Oro: Todo lo que sale de tu app puede fallar

```
async function obtenerDatos() {  
  try {  
    const res = await fetch('url');  
    // El Camino Feliz (Happy Path)  
  } catch (error) {  
    // La red de seguridad (Safety Net)  
    console.error('Algo falló:', error);  
  }  
}
```

Camino Feliz: Ejecutamos el código asumiendo que el servidor responderá correctamente.

Red de Seguridad: Atrapa automáticamente cualquier Promesa Rechazada (caída de red, error 500) evitando que la app colapse.

El Puente al Mundo Exterior: HTTP GET y Fetch



fetch() es la herramienta nativa del navegador para solicitar recursos a través de la red. Por defecto, siempre realiza una petición de tipo GET (solo lectura).

Deconstruyendo una Petición Fetch

Pausa la función hasta que el servidor responda.

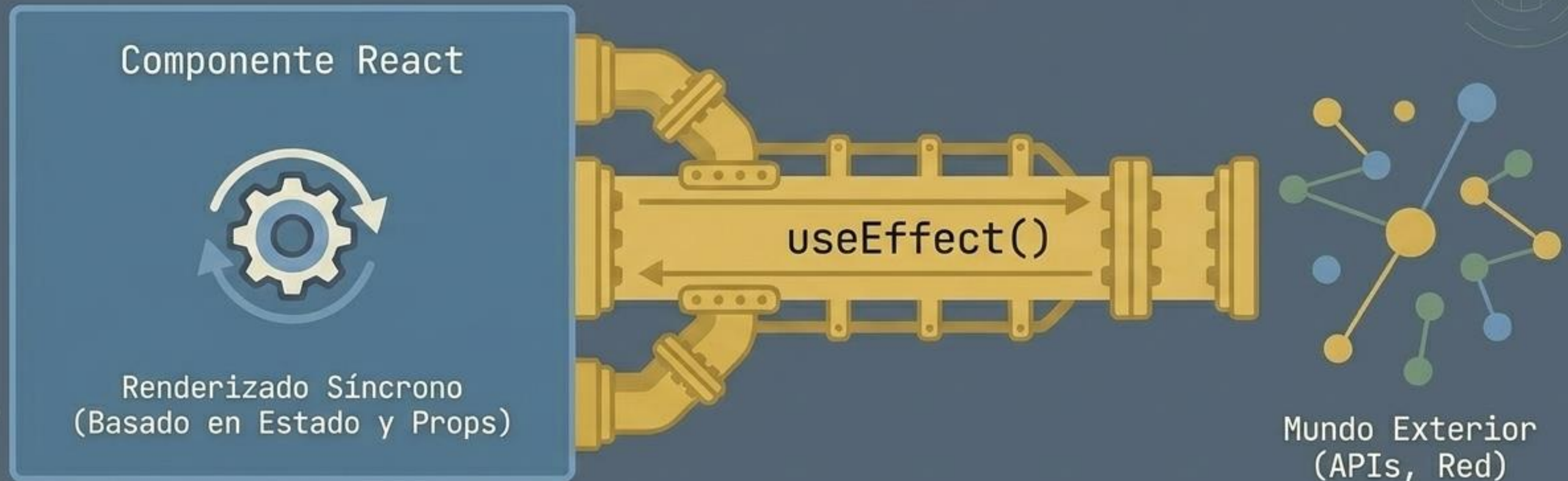
Objeto que contiene headers y status (ej. 200 OK, 404 Not Found).

```
const response = await fetch('https://api.ejemplo.com/data');  
const data = await response.json();
```

Genera una Promesa en estado Pending. Realiza un HTTP GET por defecto.

Otra operación asíncrona. Desempaqueta la respuesta y la transforma en un objeto JS.

Transición a React: El Reto de la Sincronización



React dibuja interfaces de forma inmediata y síncrona. Para poblar esa interfaz con datos asíncronos externos sin bloquear el renderizado, debemos usar la escotilla de escape llamada **Efectos Secundarios** (`'useEffect'`).

El Patrón de Arquitectura: Efecto + Estado Asíncrono



El componente siempre se dibuja primero (loading). Los datos llegan después y actualizan la vista a través del Estado.

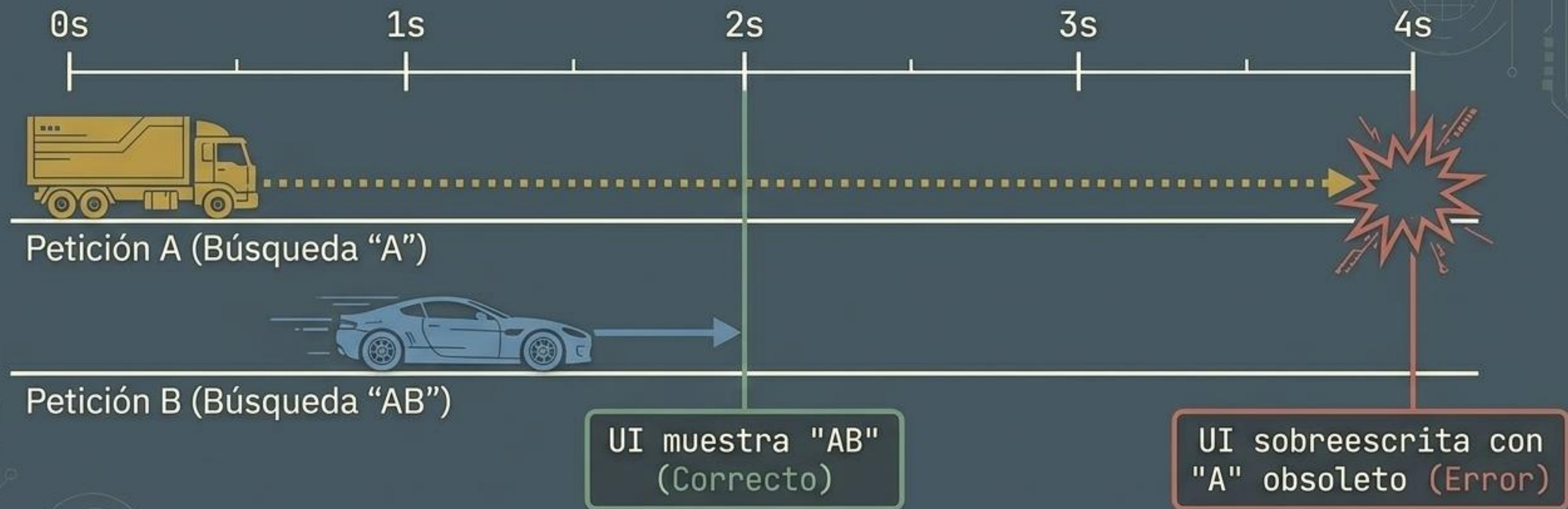
Codificando el Patrón: Fetch en React

```
useEffect(() => {  
  // 1. Declarar función DENTRO del efecto  
  async function cargarDatos() {  
    try {  
      const res = await fetch('/api/datos');  
      const data = await res.json();  
      setEstado(data); // 3. Actualizar estado  
    } catch (error) {  
      setError(true);  
    }  
  }  
}  
  
  cargarDatos(); // 2. Invocarla inmediatamente  
}, []);
```



¡Atención!
Nunca hagas que la función principal de useEffect sea async directamente. React espera que devuelva una función de limpieza, no una Promesa.

El Peligro Oculto: Condiciones de Carrera (Race Conditions)



La red es impredecible. Una petición antigua y lenta puede llegar después de una reciente, sobrescribiendo tu estado con datos obsoletos.

La Solución Elegante: Funciones de Limpieza

```
useEffect(() => {  
  let ignore = false; // Bandera de control  
  
  async function startFetching() {  
    const result = await fetchBio(person);  
    if (!ignore) {  
      setBio(result); // Solo actualiza si no se ha desmontado  
    }  
  }  
  startFetching();  
  
  return () => {  
    ignore = true; // Función de Limpieza (Cleanup)  
  };  
}, [person]);
```

Al desmontar el componente o cambiar la dependencia, React ejecuta el cleanup, invalidando la respuesta asíncrona de la petición anterior.

Matriz de Síntesis: Fetching Profesional en React



SÍ (Haz esto)

Usar try/catch para manejar caídas de API.

Proveer un estado visual de "Cargando...".

Declarar la función async dentro del bloque useEffect.

Usar una variable ignore en la función de limpieza.



NO (No hagas esto)

Dejar la interfaz congelada con peticiones síncronas.

Escribir `useEffect(async () => {...})` directamente.

Asumir que el servidor siempre devolverá un 200 OK.

Mutar el DOM directamente con el resultado en lugar de usar `setState`.

Estás Listo para Consumir Datos Reales



Event Loop y Promesas comprendidos.



Dominio de sintaxis async/await y fetch.



Sincronización con el ciclo de vida de React.

Siguiente Paso: Abre tu editor. Es hora de implementar operaciones CRUD consumiendo una API pública para tu aplicación web.

